EN3160 – Fitting and Alignment

Assignment 2: Expanded Technical Report

Student: Dilhara D.S. **Index:** 230147L

Repository: https://github.com/sachi1645/Fitting-and-Alignment

---

1 Question 1: Scale-normalised Laplacian-of-Gaussian blob detection

1.1 Objective and theory

The task is to detect approximately circular structures in the supplied sunflower field image using Laplacian-of-Gaussian (LoG) filtering and scale-space extrema, *without* the Hough-circle detector. The image is loaded through `cv2.IMREAD_REDUCED_COLOR_4`; the reported dimensions after decoding are 360×360 pixels. Pixel coordinates and reported radii below refer to this downsampled image.

Let $I(x, y)$ denote its grayscale intensity and G_σ a two-dimensional Gaussian kernel. The scale-normalised LoG response used in the implementation is

$$R(x, y, \sigma) = \left| \sigma^2 \nabla^2 (G_\sigma * I)(x, y) \right|. \tag{1}$$

The σ^2 factor supports comparisons across scales; the absolute value includes responses of either polarity. For an ideal blob, the associated characteristic circle radius is approximated by

$$r \simeq \sqrt{2}\,\sigma. \tag{2}$$

This is a *scale-derived characteristic radius*, not a geometric measurement of the exact flower-petal boundary. In particular, a dark flower centre surrounded by bright petals may yield a blob radius nearer the central disc than the full blossom diameter.

1.2 Method and parameter selection

The script evaluates 32 geometrically spaced Gaussian scales from $\sigma = 0.5$ to 8.0 reduced-image pixels. A candidate is retained if its absolute response is a local maximum in a $3 \times 7 \times 7$ scale–row–column neighbourhood and is at least 0.06. The detector rejects endpoints of the scale grid because a symmetric scale comparison is not possible there. Consequently the *tested* range is $[0.5, 8.0]$ px but the *eligible interior* extrema are approximately $\sigma \in [0.547, 7.316]$ px, corresponding to characteristic radii of roughly 0.773–10.346 px.

Detected centres are constrained to $y \geq 180$ to exclude sky and much of the tree line. Candidate circles touching image boundaries are excluded. A post-processing step further requires that an annulus near the candidate contains at least 28% yellow pixels and that the average intensity of the annulus exceeds the central intensity by at least 8 grayscale levels. Non-maximum suppression also removes nearby, similarly scaled detections.

Table 1: Question 1 algorithm parameters. Thresholds are empirical choices, not theoretically universal constants.

Image decoding	Reduced-colour image, 1/4 linear resolution
Gaussian scales	32, logarithmically spaced from 0.5 to 8.0 px
LoG response threshold	0.06 after intensity normalisation to $[0, 1]$
Extremum neighbourhood	3 scales by 7×7 pixels
Region of interest	$y \geq 180$ in reduced-image coordinates
Yellow fraction in annulus	At least 0.28
Annulus minus centre intensity	At least 8 gray levels
Geometric radius	$r = \sqrt{2}\,\sigma$

1.3 Detection output and largest candidate circles

Figure 1: Sunflower photograph with LoG-derived candidate circles.

Table 2: Eight largest reported candidates, ordered by estimated radius. Positions and radii are in reduced-image pixels.

Rank	Centre x	Centre y	σ (px)	r (px)	LoG response
1	282	338	7.32	10.35	0.349
2	79	258	6.69	9.46	0.178
3	106	255	6.12	8.65	0.362
4	179	327	6.12	8.65	0.345
5	178	261	5.59	7.91	0.353
6	339	206	5.59	7.91	0.098
7	29	272	5.12	7.23	0.151
8	189	240	4.68	6.62	0.362

For example, candidate 1 has $\sigma \simeq 7.32$ px and hence $r \simeq \sqrt{2}(7.32) = 10.35$ px; expressed in original-image pixels, its approximate centre is $(1128, 1352)$ and radius is 41.4 px (assuming exactly fourfold downsampling). The displayed centres are in image coordinate order (x, y), with y increasing downwards.

1.4 Interpretation and limitations

The results establish that the method detects candidate circular dark-centre/bright-ring structures within the flower field and avoids the prohibited Hough-circle method. The reported count of 271 is a count of *validated candidates*, not a verified count of individual sunflowers. Small overlapping flowers may be missed; colour thresholds can reject shadowed or pale flowers, and background texture can cause false positives. Since the upper-scale endpoint is excluded, large objects beyond about 10.35 reduced-image pixels in characteristic radius can also be missed.

A more defensible accuracy study would annotate a small representative subset of flowers, match candidate centres to those annotations, and report precision and recall. Showing only the 15–30 largest detections as a second clean figure would help assess whether the largest reported circles actually correspond to flower heads. The earlier report's claim that a 0.5-px lower limit *recovers* flowers missed by an earlier 1.3-px experiment should not be treated as demonstrated without a matched before/after comparison.

2 Question 2: Sequential RANSAC line and circle fitting

2.1 Synthetic data and ground truth

The supplied task specifies a noisy mixed point set with $N = 100$ samples: 50 points generated near a circle with ground-truth centre $\mathbf{c}_{\text{gt}} = (2, 3)$, radius $r_{\text{gt}} = 10$, and independent radial noise of standard deviation $10/16 = 0.625$; and 50 points generated along $y = -x + 2$ with vertical Gaussian noise of standard deviation 1. The reviewed code uses random seed 0 for reproducible data generation.

2.2 (a) Line RANSAC with a unit normal

Write the line in normal form,

$$ax + by = d, \qquad a^2 + b^2 = 1. \tag{3}$$

Two distinct sampled points $\mathbf{p}_1 = (x_1, y_1)$ and $\mathbf{p}_2 = (x_2, y_2)$ define tangent $\mathbf{t} = (\mathbf{p}_2 - \mathbf{p}_1)/\|\mathbf{p}_2 - \mathbf{p}_1\|$. A unit normal is $\mathbf{n} = (-t_y, t_x)$, with $d = \mathbf{n}^{\mathsf{T}}\mathbf{p}_1$. The appropriate error to minimise or threshold is the *perpendicular* distance

$$e_i^{\text{line}} = |ax_i + by_i - d|, \tag{4}$$

which requires no additional denominator because $\|\mathbf{n}\|_2 = 1$. The implementation performs 2,500 two-point random trials, accepts residuals up to 1.7 units, and requires at least 35 inliers. It selects the hypothesis with the largest consensus (breaking count ties by total inlier squared error), then refits the line on that set by total least squares (SVD).

For perspective, the generator's line noise is vertical with standard deviation 1, but its perpendicular component relative to a slope--1 line is $1/\sqrt{2} \approx 0.707$. A 1.7-unit perpendicular threshold is therefore approximately 2.40 times that nominal inlier-noise scale. The *minimum consensus of 35* is a design heuristic; it is not the observed line-inlier count.

2.3 (b) Circle RANSAC on the remaining points

After removing the estimated line consensus, each RANSAC trial samples three non-collinear points. The three-point circle hypothesis has centre $\mathbf{c} = (c_x, c_y)$ and radius $r > 0$. The radial residual is

$$e_i^{\text{circ}} = \left| \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2} - r \right|. \tag{5}$$

The code runs 4,500 circle trials, applies radial threshold 1.25, and requires at least 32 consensus points. The threshold is twice the generator's nominal circle radial-noise standard deviation ($1.25/0.625 = 2$). The best circle is subsequently refined over its consensus by numerical minimisation of

$$J(c_x, c_y, r) = \sum_{i \in \mathcal{I}_{\text{circle}}} \left(\|\mathbf{x}_i - \mathbf{c}\|_2 - r \right)^2, \tag{6}$$

followed by inlier reassignment and one further refinement. This is different from fitting a circle directly to all 100 points: only the selected consensus contributes to the final geometric fit.

2.4 (c) Results, samples and fitted models

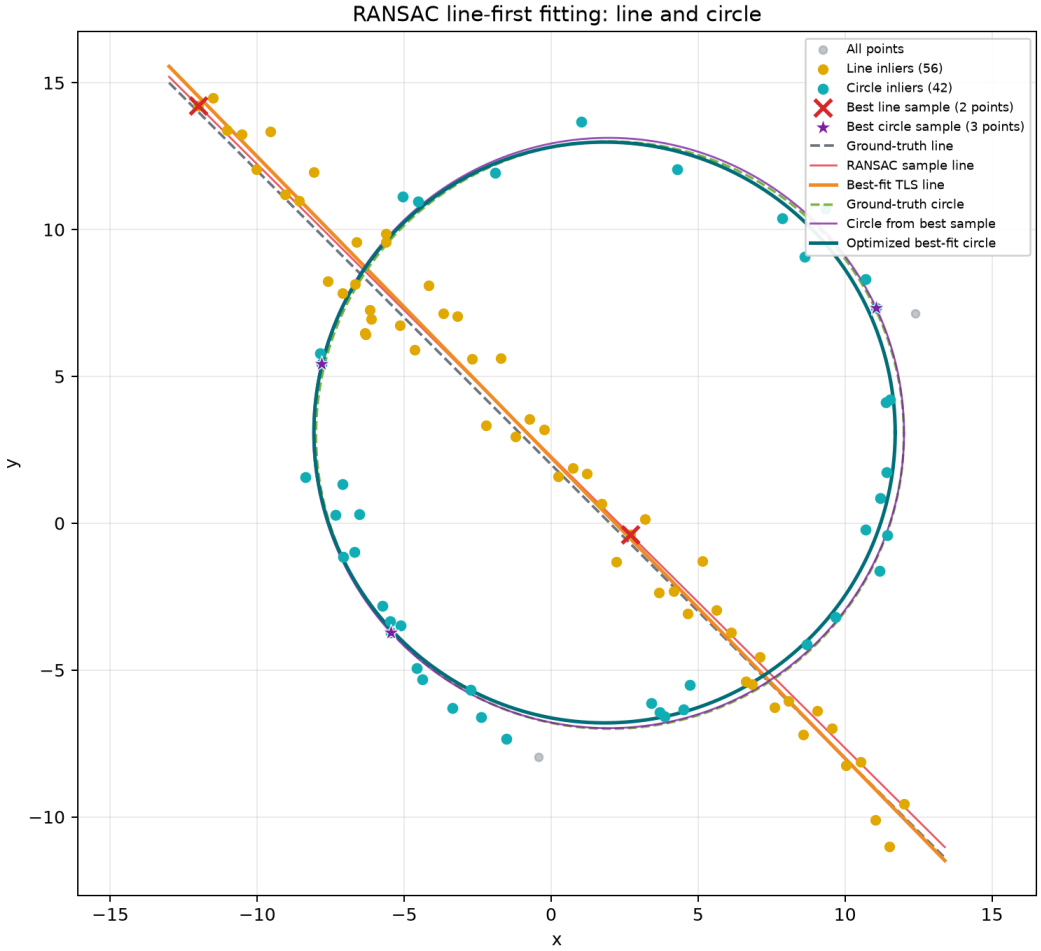


Figure 2: Reproduced plot of all 100 points, the best minimal line and circle samples, corresponding sample models, final refined models, model inliers and ground-truth line/circle.

Table 3: Numerical results reproduced by executing `q2_ransac_fitting.py -seed 0`.

Quantity	True model	Estimated model
Line unit normal $[a, b]$	$[0.707107, 0.707107]$	$[0.715649, 0.698460]$
Line normal 2-norm	1	1.000000
Normal-form offset d	1.414214	1.561775
Line slope m	-1.000000	-1.024609
Line intercept b	2.000000	2.236025
Circle centre (c_x, c_y)	$(2, 3)$	$(1.817367, 3.089484)$
Circle radius r	10	9.888575

The detected line consensus is 56/100 points, comprising all 50 generated line points and six generated circle points lying within the line threshold. Removing those leaves 44 points. The final circle consensus has 42 points, all belonging to the generated circle class. Thus, two points remain unassigned after these two estimated consensuses. The circle centre error is $\|\widehat{\mathbf{c}} - \mathbf{c}_{\text{gt}}\|_2 \approx 0.2034$ (*not* 0.204 at three-decimal rounding), and the absolute radius error is $|9.888575 - 10| \approx 0.1114$. Using the known synthetic labels to evaluate class composition is legitimate here; an unknown real dataset would not provide that ground truth.

2.5 (d) What if the circle is fitted first?

The same code also executes the reverse sequence: fit the circle to all 100 noisy points, remove its consensus, and then fit the line to the remainder. This experiment returns 57 circle-consensus points (48 genuine circle points *and nine line points*). It leaves 43 points, from which the subsequent line fit selects 41 genuine line points. Hence the first-estimated structure can claim nearby observations generated by the other structure.

Sequential fitting is therefore order dependent. This example does *not* prove that a line-first order is always better: model support, intersection geometry, noise, error thresholds, and sampling seeds all matter. Simultaneous multi-model fitting, post-hoc reassignment, or iterative refinement could reduce order bias. For this dataset, the stated line-first processing agrees with the assignment and produces a good approximation of both ground-truth models.

3 Question 3: Four-point homography and perspective compositing

3.1 Projective geometry and procedure

For a point on a planar overlay, the homography maps homogeneous source pixel coordinates $\mathbf{x} = (x, y, 1)^{\mathsf{T}}$ to destination coordinates $\mathbf{x}' \sim H\mathbf{x}$, where

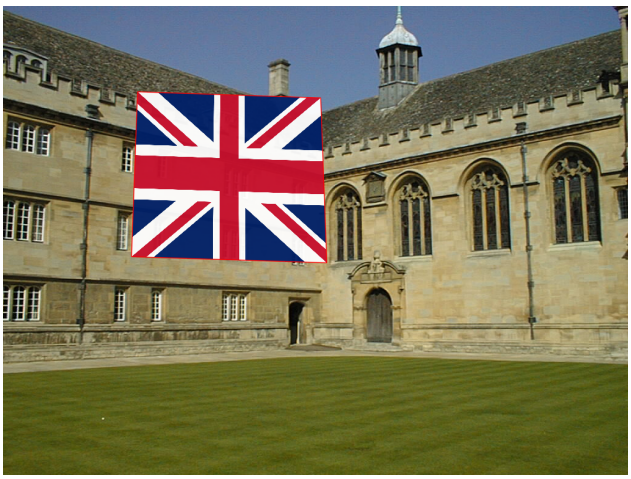
$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}, \quad x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}. \tag{7}$$

Four ordered correspondences, provided that the configuration is non-degenerate, are sufficient to determine the eight independent degrees of freedom of H up to a nonzero scale. The code supplies the full overlay image corners in the order top-left, top-right, bottom-right, bottom-left and four destination points in the same cyclic order, calls `cv2.getPerspectiveTransform`, and warps using `cv2.warpPerspective`. A similarly warped binary mask restricts alpha blending to the polygon region:

$$I_{\text{result}}(x, y) = \alpha I_{\text{warped}}(x, y) + (1 - \alpha) I_{\text{background}}(x, y), \qquad (x, y) \in \Omega, \tag{8}$$

while pixels outside the valid warped mask Ω remain unchanged. The tested alpha value is $\alpha = 0.97$.

3.2 Reported experiments



(a) Union Jack over the Wadham College facade.

(b) Sunflower photograph placed on the same facade.

Table 4: Source/destination choices and alpha for the two reported examples. Coordinates follow TL, TR, BR, BL order.

Field	Example 1	Example 2
Background	`wadham.jpg`	`wadham.jpg`
Overlay	`flag.png`	Sunflower-field photograph
Destination points	$(220, 250), (480, 277), (474, 410), (215, 394)$	$(72, 295), (185, 303), (181, 382), (69, 374)$
Blend parameter	$\alpha = 0.97$	$\alpha = 0.97$

3.3 Assessment and required improvement

The outputs visibly demonstrate planar image warping, the importance of consistent corner ordering, and masked compositing. The approach assumes the selected target surface is planar and thus cannot model depth changes, occluding components or local parallax. A red one-pixel outline is added in the provided code *after* compositing; it is useful for checking target geometry but should be turned off in a photorealistic final composite.

4 Question 4: SIFT-based PCB alignment and difference analysis

4.1 Feature detection, matching and geometric alignment

The template is `AECC1.jpg`; the test images are `AAAA1`, `ADAD1`, `AECC2`, `AECC3` and `AECC4`. In each pair, the code (i) computes SIFT keypoints and 128-element descriptors for the test image and template, (ii) obtains nearest-neighbour matches using `cv2.BFMatcher(cv2.NORM_L2)`, (iii) retains matches satisfying Lowe's ratio test with threshold 0.75, and (iv) estimates H from test-board coordinates to template coordinates using `cv2.findHomography(..., cv2.RANSAC, 5.0)` with a five-pixel reprojection threshold. The given image is then warped to the template's pixel dimensions.

The script also warps a full-image validity mask. Pixels with no corresponding warped input are excluded from its reported difference statistics; the code does *not* isolate the PCB itself from its surrounding photograph. Thus a fairer comparison among test images would additionally use a common board-region mask and report the fraction of the template actually covered.

4.2 Exactly what difference measure is reported?

This distinction matters because the original report's label *"absolute grayscale difference"* is not quite correct. The supplied Python computes

```
diff = cv2.absdiff(template, aligned) # absolute B/G/R channel differences
diff_gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)
diff_gray[~valid] = 0
```

For a BGR image this corresponds approximately to the *grayscale-weighted sum of absolute channel differences*

$$D_{\text{code}} = 0.114|B_t - B_a| + 0.587|G_t - G_a| + 0.299|R_t - R_a|, \tag{9}$$

not, in general, $|0.114(B_t - B_a) + 0.587(G_t - G_a) + 0.299(R_t - R_a)|$. Both are reasonable photometric comparison measures, but the report must name the one the code actually uses. To calculate a conventional absolute grayscale-image difference instead, use:

```
t_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
a_gray = cv2.cvtColor(aligned, cv2.COLOR_BGR2GRAY)
diff_gray = cv2.absdiff(t_gray, a_gray)
diff_gray[~valid] = 0
```

Do not reuse the following numerical measurements after making this code change; rerun the original images and regenerate the table. In this reviewed edition, all numerical measurements are preserved from the actual implemented metric in Eq. (9).

4.3 Reported matching and differencing results

Table 5: Values are in pixel-intensity units on $[0, 255]$.

Image	Ratio matches	RANSAC inliers	Inlier rate	Mean D_{code}	95th percentile
AAAA1	51	18	35.3%	28.8	107
ADAD1	369	194	52.6%	31.2	92
AECC2	4322	3847	89.0%	3.5	9
AECC3	5629	5006	88.9%	3.2	8
AECC4	3830	3402	88.8%	4.0	10

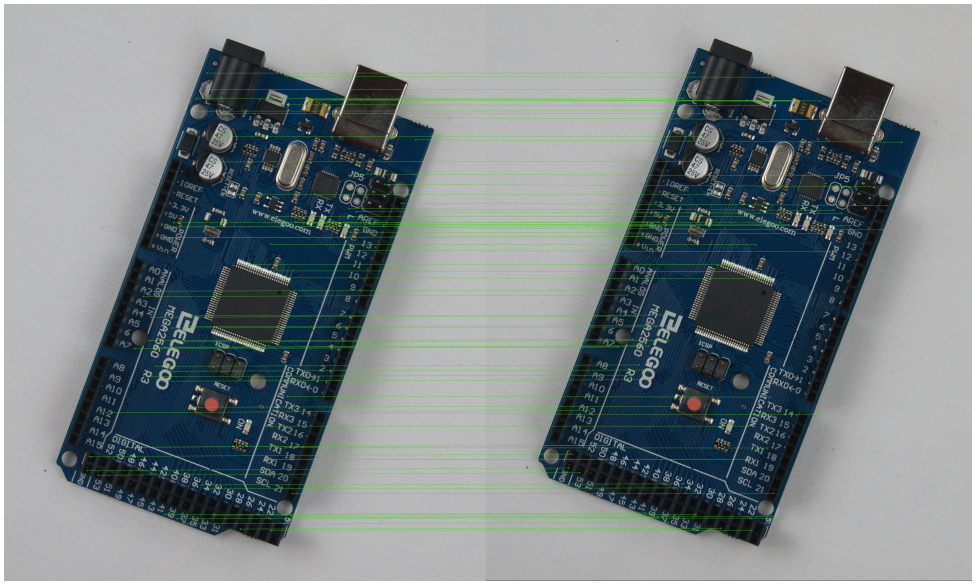
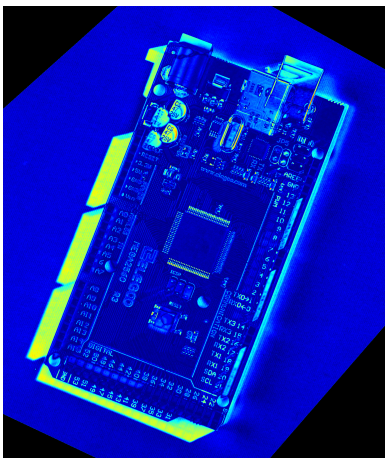
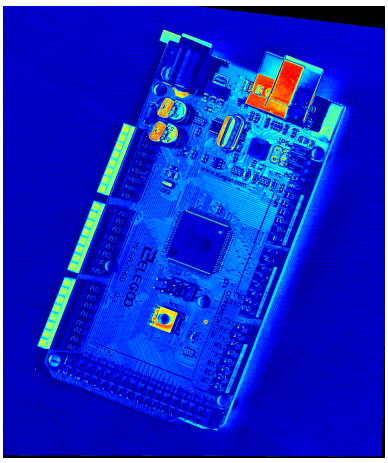


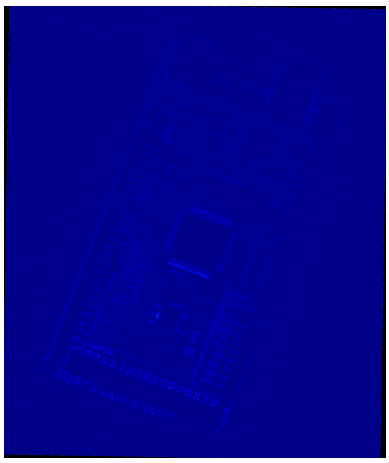
Figure 4: Reported SIFT/RANSAC inlier-match visualization for AECC2 versus the AECC1 template. Only one match figure is present in the original report; figures for other test images can strengthen part (b).



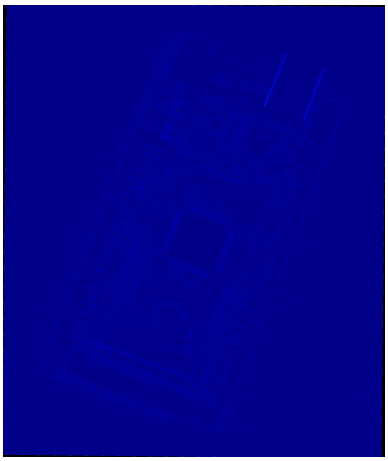
(a) AAAA1



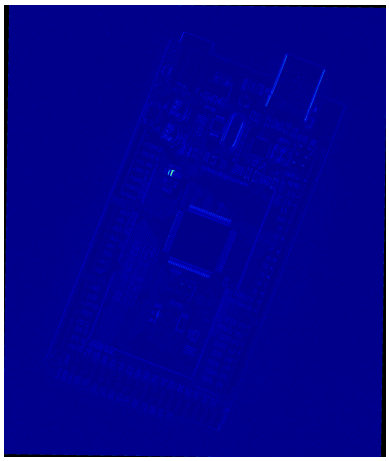
(b) ADAD1



(c) AECC2



(d) AECC3



(e) AECC4

Figure 5: Difference heatmaps for each test image.

4.4 Discussion of individual test images

AECC2–AECC4: Their high RANSAC-inlier fractions (about 89%) and much smaller reported mean differences (3.2–4.0 units) are consistent with comparatively good geometric and photometric agreement with AECC1 over the measured regions. Small residual differences can arise from changes in lighting and colour, small positional offsets, interpolation during `warpPerspective`, noise, and physically varying components.

AAAA1 and ADAD1: Their lower inlier fractions (35.3% and 52.6%) and markedly higher mean differences (28.8 and 31.2) show that they are substantially less similar to the template under this procedure. Such differences could reflect a different PCB revision or component placement, weaker geometric registration, or capture/illumination differences. In particular, 18 inliers for AAAA1 is a relatively small geometric support set, so its transformation should be inspected for plausibility before interpreting bright areas as true hardware changes.

Why do USB Type-B connectors, capacitors and reset buttons show high differences? A homography models a single plane, approximately the board surface. Tall parts sit above that plane; viewing-angle differences create parallax, leaving those parts locally misregistered after the board plane is aligned. Connector and capacitor geometry may genuinely vary between boards or revisions; highlights and shadows from raised components are also illumination-sensitive. Therefore, high local error can represent real hardware differences *or* unavoidable geometric/photometric mismatch, not automatically defects.

4.5 How to improve the reliability of the experiment

For each of the five test boards, add a three-column figure showing *template*, *aligned test image*, and *difference*, with a separate close-up of a suspected component. Include the inlier-match panel, the estimated overlap percentage, and a consistent shared intensity scale for heatmaps. The current code draws only the 75 lowest-distance inlier matches in its preview, even when there are thousands of matches. For quantitative board-to-board comparison, use a PCB region-of-interest mask so background colours do not dominate the mean, and consider photometric normalisation before differencing. These are recommended further experiments; their results are not claimed in this report.